

Assignment #2 (Part I)

ECE 422C (Prof. Edison Thomaz) - The University of Texas at Austin - Fall 2026

Due date: Sept 22nd 2026

Important: This is Phase I of a multi-phase assignment. You are building Mastermind as the first application in a small family of interactive games. Part II will require you to reuse and adapt this architecture for another game. The identity of that game will not be announced until Phase I closes.

Learning objectives

- Design a nontrivial Java program using classes, interfaces, encapsulation, composition, and polymorphism where appropriate.
- Separate game-independent infrastructure from game-specific rules and representations.
- Translate requirements into responsibilities and collaborations among objects.
- Build an architecture that can evolve when requirements change.
- Explain and defend design decisions, including decisions suggested by AI tools.

AI policy

AI tools are permitted. You may use them for brainstorming designs, generating candidate code, debugging, testing, documentation, and learning Java APIs. You remain responsible for the architecture and every line you submit. During checkout you may be asked to explain, critique, or extend your design. A design is not justified merely because an AI tool proposed it.

Phase I task: Mastermind

Implement a text-based version of Mastermind. Preserve the core behavior of the previous assignment while treating this as an architecture exercise rather than merely an output-matching exercise.

Mastermind rules

- The computer generates a secret code.
- The default code contains 4 colored pegs.
- Default colors are B, G, O, P, R, and Y. Colors may repeat.
- The player has 12 valid guesses by default.
- A black peg means correct color in the correct position.
- A white peg means correct color in the wrong position.
- Each peg in the secret and guess may contribute to feedback at most once; exact matches take precedence.
- Invalid guesses do not consume an attempt.
- Typing HISTORY displays all valid guesses and their feedback in order.
- After a win or loss, the user can choose whether to play another game.
- A testing mode must make it possible to reveal or deterministically control the secret for testing.

Configuration

The following must be changeable without rewriting the game algorithm:

- maximum number of guesses
- number of pegs in the code
- set of legal colors (up to 10 distinct one-character symbols)

You may use a `GameConfiguration` class or another clearly isolated configuration mechanism.

Console interaction

The program must provide clear prompts, report invalid guesses, report feedback in the form `nB_mW`, support HISTORY, and report win/loss. Exact prose and whitespace do not need to match the old assignment character-for-character unless a Gradescope test explicitly specifies an output token. The focus of this assignment is architecture.

Required launch point:

```
java assignment2.Driver mastermind [test]
```

Architecture requirement

Your design must distinguish between concepts that are intrinsic to Mastermind and concepts that could plausibly support another turn-based guessing game. You are not expected to predict the future perfectly. You are expected to make deliberate choices.

Do NOT create classes or interfaces merely to increase the class count. An abstraction should have a clear responsibility or enable meaningful substitution/reuse.

Only Driver is prescribed. You may choose the remaining class/interface names and organization. In particular, there is no required Game class or required `runGame()` method.

Required deliverables

1. A working Mastermind implementation under package `assignment2`.
2. A `Design.pdf` document containing a class/interface diagram, the responsibility of each major type, which pieces you expect to reuse in another game, which pieces are intentionally Mastermind-specific.
3. A `Tests.pdf` describing at least 8 meaningful tests, including duplicate-color feedback cases, invalid input, history, win/loss, and at least one changed configuration.
4. An `AI.pdf` listing AI tools used, up to three important AI interactions, and one AI-generated design/code suggestion you changed or rejected. If you used no AI, state that and discuss one design you reconsidered.
5. A ZIP named `Project2_Phase1_EID.zip` with the code and pdfs.

Upload the zip file to both Canvas AND Gradescope.

Phase I is graded before Phase II is released. Keep the exact Phase I version in source control or a separate copy; you will need it for the architecture-change report.